

PROJECT REPORT

LIBRARY MANAGEMENT SYSTEM

CSE1021 – Introduction to Problem Solving and Programming

Student Name	Sunkavalli Hema Durga vara chowdary
Registration No.	26BAI10891
Teacher	Chayan Paul
Project Type	Build Your Own Project

Submitted as part of the course project requirements.

1. Introduction

The Library Management System is a Python-based console application developed to manage basic library operations. It provides facilities for adding and viewing books, registering and viewing members, issuing and returning books, searching for books, and generating a library report. Data is stored in JSON files so that records can be reused when the program is opened again.

The project applies problem-solving and programming concepts from CSE1021, including functions, modules, conditional statements, loops, lists, dictionaries, input validation, file handling, and modular program design.

2. Problem Statement

Managing books and members manually can take more time and can lead to errors in maintaining records. The proposed system provides a simple computerized approach for maintaining basic library records and performing common library operations.

3. Objectives

- Manage library books in an organized way.
- Maintain member details.
- Issue and return books.
- Search for books by title or author.
- Generate a basic library status report.
- Persist book and member data using JSON files.
- Apply modular Python programming and input validation.

4. Functional Requirements

- Add books with a unique book ID, title, author, and availability status.
- View the stored list of books.
- Register members with a unique member ID, name, and phone number.
- View registered members.
- Issue a book to a valid member when the book is available.
- Return an issued book.
- Search for a book using its title or author.
- Generate a report showing total, available, and issued books and total members.
- Save and load book/member records using JSON files.
- Validate required user input.

4.1 Non-Functional Requirements

Usability	Simple menu-driven console interface.
------------------	---------------------------------------

Reliability	Checks for missing records, duplicate IDs, and invalid input.
Maintainability	Features are separated into independent Python modules.
Error Handling	User-friendly messages are displayed for invalid operations.
Resource Efficiency	The system uses lightweight JSON storage and standard Python modules.

5. Technologies and Tools

- Python – primary programming language.
- JSON – persistent storage for books and members.
- VS Code – development environment.
- Git and GitHub – version control and project submission.

6. System Architecture

The system follows a modular architecture. The user interacts with `main.py`, which acts as the main controller and calls the appropriate modules. The feature modules communicate with `storage.py` for JSON file handling, while `validation.py` provides reusable input validation.

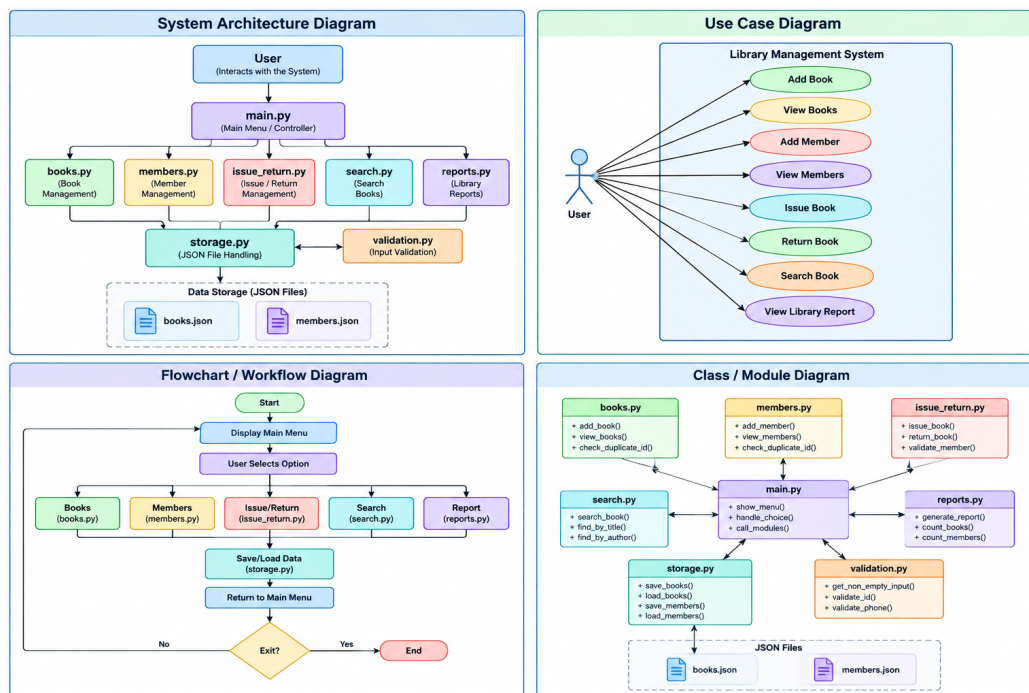


Figure 1. System architecture, use cases, workflow and module design.

7. Design Diagrams

7.1 Use Case Diagram

The main actor is the User. The available use cases are Add Book, View Books, Add Member, View Members, Issue Book, Return Book, Search Book, and View Library Report.

7.2 Workflow Diagram

The workflow begins with the main menu. The user selects an operation, the corresponding module performs the operation, data is saved or loaded as required, and control returns to the main menu until the user selects Exit.

7.3 Class / Module Diagram

The project is organized into eight functional Python modules: main.py, books.py, members.py, issue_return.py, search.py, reports.py, storage.py, and validation.py. This separation keeps the program easier to understand and maintain.

7.4 Storage Design

The application uses two JSON files: books.json for book records and members.json for member records. The storage module creates the data folder when required and loads/saves JSON data.

8. Implementation Details

Module	Responsibility
main.py	Displays the menu and controls program flow.
books.py	Adds and displays books and checks duplicate book IDs.
members.py	Adds and displays members and checks duplicate member IDs.
issue_return.py	Issues and returns books after validating records and status.
search.py	Searches books using title or author keywords.
reports.py	Calculates total, available, and issued books and total members.
storage.py	Loads and saves books and members using JSON.
validation.py	Provides reusable input validation functions.

8.1 Program Flow

main.py presents a nine-option menu. Based on the selected option, it calls the relevant function from the books, members, issue_return, search, or reports module. After completion, the program returns to the menu. Selecting Exit terminates the loop.

8.2 Data Storage

storage.py uses Python's json and os modules. It creates the data folder if it does not exist and saves or loads records from books.json and members.json.

9. Testing Approach

Test Case	Expected Result
Add a new book	Book is added and saved.
Add duplicate book ID	System rejects the duplicate ID.
Add a member	Member is added and saved.
Issue an available book	Book status changes to Issued.
Issue a missing book/member	Appropriate error message is displayed.
Return an issued book	Book status changes to Available.
Search by title/author	Matching books are displayed.
Generate report	Book/member counts are displayed.
Enter empty required input	Validation requests a non-empty value.

10. Results

The completed application provides a working menu-driven library management workflow. The project supports book and member management, issuing and returning books, searching, report generation, JSON persistence, and input validation.

11. Challenges Faced

- Designing the project as separate modules while keeping the modules connected correctly.
- Maintaining consistent book and member data across different operations.
- Handling invalid input and preventing duplicate IDs.
- Implementing persistent JSON storage without requiring a database.

12. Learnings and Key Takeaways

- Improved understanding of Python functions and modules.
- Applied lists and dictionaries to represent structured data.
- Used loops and conditional statements to implement application logic.
- Learned basic file and JSON handling for persistent data.
- Understood the value of modular design and input validation.
- Applied problem-solving concepts to a real-world style application.

13. Future Enhancements

- Store complete issue/return transaction history.
- Add due dates and overdue tracking.
- Add a graphical user interface.
- Use a relational database for larger datasets.
- Add login/authentication and role-based access.

- Provide richer reports and export options.

14. Conclusion

The Library Management System demonstrates how Python programming and problem-solving concepts can be combined to build a practical application. Its modular structure separates major responsibilities and makes the project easier to understand and maintain. The system provides the core functionality required for basic library management while leaving clear possibilities for future enhancement.

15. References

1. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.
2. CSE1021 – Introduction to Problem Solving and Programming course syllabus.
3. Python documentation and standard-library concepts used in the implementation.